

TÀI LIỆU TRA LỜI CÂU HỎI HỘI ĐỒNG

Đề tài: *Insightimate – Nâng cao độ chính xác ước lượng nỗ lực phần mềm*
sử dụng Machine Learning theo ba schema: LOC, FP, UCP

Hội nghị Sinh viên Nghiên cứu Khoa học 2026 – Đại học Duy Tân

Ngày: 23/04/2026 | Phiên bản: 2.0 (Mở rộng theo đề cương AI Module)

Hướng dẫn sử dụng: Tài liệu này tổng hợp các câu hỏi dự kiến từ hội đồng phản biện, bao gồm cả câu hỏi kỹ thuật chuyên sâu về COCOMO II, tiền xử lý dữ liệu, mô hình ML và hướng mở rộng hệ thống. Nhóm nên đọc kỹ từng câu trả lời và luyện tập nói trước gương.

NHÓM 1 – Tổng quan về đề tài & COCOMO II

[C1] COCOMO II là gì? Tại sao nhóm chọn COCOMO II làm nền tảng lý thuyết?

Trả lời: COCOMO II (Constructive Cost Model II) là mô hình ước lượng chi phí và nỗ lực phát triển phần mềm do Barry Boehm và cộng sự phát triển (1995, USC). COCOMO II cải tiến so với COCOMO I bằng cách hỗ trợ nhiều vòng đời phát triển hiện đại (Agile, OO, reuse-based).

Công thức cốt lõi:

$$\text{Effort} = A \times \text{Size}^E \times \prod_{i=1}^n EM_i$$

trong đó A là hằng số hiệu chỉnh, E là số mũ quy mô (scale factor), EM_i là các hệ số điều chỉnh nỗ lực (effort multipliers).

Lý do chọn:

- Được sử dụng rộng rãi nhất trong nghiên cứu học thuật và thực tế công nghiệp.
- Hỗ trợ cả ba metric kích thước: LOC, FP, UCP – phù hợp với ba schema của đề tài.
- Có nền tảng toán học rõ ràng, dễ mở rộng tích hợp ML.
- Nguồn tham chiếu: Boehm et al. (2000), *Software Cost Estimation with COCOMO II*, Prentice Hall.

[C2] Sự khác biệt giữa COCOMO I và COCOMO II là gì?

	Tiêu chí	COCOMO I	COCOMO II
Trả lời:	Năm ra đời	1981	1995–2000
	Metric kích thước	Chỉ LOC	LOC, FP, UCP
	Vòng đời hỗ trợ	Waterfall	Waterfall, Agile, Reuse
	Số hệ số điều chỉnh	15	17 (EM) + 5 (SF)
	Hiệu chỉnh	Cố định	Có thể hiệu chỉnh theo tổ chức

[C3] Ba schema LOC, FP, UCP là gì? Tại sao đề tài nghiên cứu cả ba schema thay vì chỉ một?

Trả lời:

- **LOC (Lines of Code / KLOC):** Đo kích thước dựa trên số dòng code. Dễ đo, nhưng phụ thuộc ngôn ngữ lập trình. Dataset: 947 mẫu.
- **FP (Function Points):** Đo chức năng phần mềm độc lập với ngôn ngữ. Chuẩn ISO/IEC 20926. Dataset: 24 mẫu.
- **UCP (Use Case Points):** Đo dựa trên các use case, phù hợp phát triển hướng đối tượng (OOP/UML). Dataset: phụ thuộc nguồn thu thập.

Lý do nghiên cứu cả ba: Mỗi tổ chức/dự án sử dụng metric khác nhau. Việc hỗ trợ cả ba giúp hệ thống *Insightimate* phục vụ được nhiều loại dự án thực tế, đồng thời cho phép so sánh hiệu quả mô hình ML trên các schema khác nhau.

[C4] Vì sao số lượng mẫu FP (24 mẫu) và UCP ít hơn LOC (947 mẫu) nhiều?

Trả lời:

- **LOC** dễ đo tự động từ mã nguồn, nên dữ liệu LOC-based phong phú hơn trong các repository công khai.
- **FP** yêu cầu chuyên gia đếm thủ công theo chuẩn IFPUG hoặc NESMA – tốn thời gian, ít tổ chức chia sẻ công khai.
- **UCP** gắn liền với tài liệu use case nội bộ – thường không công bố.

Đây cũng là lý do nhóm sử dụng **Random Forest** thay vì Deep Learning: RF hoạt động tốt với bộ dữ liệu nhỏ, không cần hàng nghìn mẫu.

NHÓM 2 – Thu thập dữ liệu

[C5] Nhóm thu thập dữ liệu từ đâu? Tiêu chí tìm kiếm như thế nào?

Trả lời: Nguồn học thuật chính: IEEE Xplore, ACM Digital Library, ScienceDirect, Springer Link, Scopus, Google Scholar, ResearchGate, arXiv.

Từ khóa chính: “COCOMO II”, “software effort estimation”, “cost estimation model”, “software project estimation”

Từ khóa phụ: “machine learning”, “agile”, “DevOps”, “calibration”, “Monte Carlo”, “risk analysis”

Kết hợp ví dụ: “COCOMO II AND machine learning”, “COCOMO II AND agile”

Ngoài ra nhóm tham chiếu các dataset benchmark nổi tiếng: ISBSG, Desharnais, Maxwell, Albrecht, Kemerer.

[C6] Làm sao nhóm đảm bảo chất lượng dữ liệu thu thập?

Trả lời: Nhóm áp dụng bộ tiêu chí sàng lọc:

1. **Kiểm tra trường quan trọng:** Mỗi mẫu phải có đủ ít nhất metric kích thước (LOC/FP/UCP) và Effort.
2. **Kiểm tra đơn vị đo:** Chỉ giữ các mẫu xác định rõ đơn vị (ví dụ: Effort = person-month, LOC = KLOC).
3. **Loại bỏ trùng lặp:** Dùng `df.drop_duplicates()` trên các trường định danh.
4. **Ghi nhận nguồn:** Thêm cột source để truy vết và kiểm định sau.

NHÓM 3 – Tiền xử lý & Chuẩn hóa Dữ liệu

[C7] Nhóm xử lý dữ liệu không đồng nhất như thế nào? Tại sao cần chuẩn hóa?

Trả lời: Dữ liệu từ nhiều nguồn thường không đồng nhất về:

- **Đơn vị đo:** LOC vs KLOC; Effort tính bằng giờ, ngày, tháng, người-tháng.
- **Tên cột:** effort, Effort_PM, actual_effort...
- **Thiếu giá trị** (missing values).
- **Giá trị ngoại lai** (outliers).

Quy trình chuẩn hóa 7 bước:

1. Phân loại schema (LOC / FP / UCP)
2. Chuẩn hóa đơn vị đo

3. Xử lý missing values
4. Xử lý outliers bằng IQR
5. Log-transform các biến lệch phân phối
6. Scaling (MinMaxScaler / StandardScaler)
7. Encoding biến phân loại

[C8] Tại sao phân loại dữ liệu theo ba schema riêng biệt?

Trả lời: Mỗi schema có:

- Phân phối đặc trưng riêng (LOC phân phối lệch phải mạnh hơn FP).
- Mỗi quan hệ Effort–Size khác nhau (hệ số E trong COCOMO II khác nhau).
- Phạm vi giá trị khác nhau.

Việc trộn các schema sẽ làm sai lệch mô hình. Ba file output: `loc_based.csv`, `fp_based.csv`, `ucp_based.csv`.

[C9] Nhóm chuẩn hóa đơn vị đo như thế nào?

	Trường	Nguồn gốc	Đơn vị chuẩn	Quy đổi
Trả lời:	LOC	7000 LOC hoặc 7 KLOC	KLOC	Chia 1000 nếu cần
	Effort	160 giờ / 20 ngày / 3 tháng	Người-tháng	1 tháng = 160 giờ = 20 ngày
	Time	Tuần, ngày	Tháng	1 tháng $\approx$ 4.3 tuần = 20 ngày
	Developers	Số nguyên	Người	Giữ nguyên

Nếu không xác định rõ đơn vị, dòng đó bị loại bỏ để tránh sai lệch.

[C10] Phương pháp IQR xử lý outliers hoạt động như thế nào? Tại sao dùng IQR thay vì Z-score?

Trả lời: Công thức IQR:

$$Q_1 = \text{quantile}(0.25) \quad (\text{phân vị thứ } 25\%)$$

$$Q_3 = \text{quantile}(0.75) \quad (\text{phân vị thứ } 75\%)$$

$$IQR = Q_3 - Q_1$$

$$\text{Lower bound} = Q_1 - 1.5 \times IQR$$

$$\text{Upper bound} = Q_3 + 1.5 \times IQR$$

Chiến lược: Dùng `clip(lower, upper)` thay vì xóa dòng – giữ nguyên số lượng mẫu, chỉ “kẹp” giá trị về ngưỡng cho phép.

Lý do chọn IQR hơn Z-score:

- Z-score giả định phân phối chuẩn (normal) – dữ liệu effort thường lệch phải (right-skewed), không thỏa mãn.
- IQR không giả định phân phối, robust với dữ liệu phi tuyến.
- Phù hợp với dataset nhỏ (FP: 24 mẫu, UCP: vài chục mẫu).

[C11] Log-transform ($\log_{1p}$) là gì? Tại sao cần thiết cho bài toán này?

Trả lời: Định nghĩa: $\log_{1p}(x) = \ln(x + 1)$, áp dụng cho các biến có phân phối lệch (skewed) như LOC, Effort, FP, UCP.

Tại sao dùng $\log_{1p}$ thay vì $\log$? $\log(0)$ không xác định; $\log_{1p}$ đảm bảo an toàn khi $x = 0$.

Lợi ích:

- Chuyển phân phối lệch phải thành gần phân phối chuẩn hơn – cải thiện hiệu quả của các thuật toán ML.
- Phù hợp với bản chất COCOMO: $\text{Effort} = A \times \text{Size}^E$ tương đương $\ln(\text{Effort}) = \ln(A) + E \cdot \ln(\text{Size})$ – mô hình mũ trở thành tuyến tính trong không gian log.
- Giảm ảnh hưởng của outliers còn sót lại sau IQR.

So sánh trước/sau: Hệ số tương quan KLOC–Effort thường tăng từ ~ 0.65 lên ~ 0.82 sau log-transform, xác nhận mối quan hệ log-linear.

[C12] Correlation analysis (phân tích tương quan) cho kết quả gì? Nhóm rút ra điều gì?

Trả lời: Phương pháp: Tính ma trận tương quan Pearson, trực quan hóa bằng heatmap (seaborn).

Kết quả điển hình (schema LOC):

- $r(\text{KLOC}, \text{Effort}) \approx 0.70\text{--}0.85$ (sau log-transform).
- $r(\text{Developers}, \text{Effort}) \approx 0.55\text{--}0.70$.
- $r(\text{Time}, \text{Effort}) \approx 0.60\text{--}0.75$.

Ý nghĩa:

- Tương quan dương: Kích thước code tăng $\rightarrow$ nỗ lực tăng (xác nhận giả thuyết COCOMO).
- Hệ số > 0.7 cho thấy mối quan hệ mạnh, đủ cơ sở huấn luyện mô hình.

- Scatter plot xác nhận mối quan hệ **phi tuyến** (mũ) hơn tuyến tính – Random Forest phù hợp hơn Linear Regression.

NHÓM 4 – Mô hình Machine Learning

[C13] Nhóm đã thử nghiệm những thuật toán ML nào? Kết quả so sánh?

Trả lời: Ba thuật toán được huấn luyện và so sánh:

- Linear Regression** – baseline, đơn giản nhất, giả định quan hệ tuyến tính.
- Decision Tree Regressor** – học phi tuyến, dễ giải thích, dễ overfitting.
- Random Forest Regressor** – ensemble (tập hợp nhiều cây), robust, ít overfitting nhất.

Metric đánh giá: MAE (Mean Absolute Error), RMSE (Root Mean Squared Error), R^2 (coefficient of determination), MMRE (Mean Magnitude of Relative Error).

Kết quả: Random Forest đạt R^2 cao nhất và MMRE thấp nhất trên cả ba schema, đặc biệt với dataset nhỏ như FP và UCP.

[C14] Tại sao không dùng Deep Learning (Neural Network) mà dùng Random Forest?

	Tiêu chí	Deep Learning	Random Forest
Trả lời:	Yêu cầu dữ liệu	>10,000 mẫu	Tốt với <1,000 mẫu
	Kích thước dataset FP	Không đủ (24 mẫu)	Phù hợp
	Khả năng giải thích	Khó (“black box”)	Có feature importance
	Thời gian training	Dài	Nhanh
	Overfitting	Cao khi ít dữ liệu	Thấp (bagging)

Kết luận: Với bộ dữ liệu nhỏ và yêu cầu giải thích được (interpretability), Random Forest là lựa chọn tối ưu. Deep Learning sẽ được xem xét trong giai đoạn thu thập thêm dữ liệu (LSTM, transfer learning – hướng tương lai).

[C15] Random Forest hoạt động như thế nào? Tại sao nó giảm overfitting?

Trả lời: Random Forest là thuật toán **ensemble** dựa trên **Bagging** (Bootstrap Aggregating):

- Tạo n tập bootstrap từ training data (lấy mẫu có thay thế).
- Huấn luyện n Decision Tree độc lập, mỗi cây chỉ dùng một tập con ngẫu nhiên của features.

3. Dự đoán cuối = trung bình (regression) của n cây.

Giảm overfitting: Vì mỗi cây học trên subset khác nhau, lỗi của các cây không tương quan – khi lấy trung bình, noise bị triệt tiêu.

Feature importance: RF cung cấp điểm quan trọng của từng feature (Gini importance), giúp xác định KLOC, FP, UCP có ảnh hưởng lớn nhất đến Effort.

[C16] Nhóm dùng phương pháp đánh giá (cross-validation) nào? Tại sao?

Trả lời: Với dataset nhỏ (đặc biệt FP: 24 mẫu), nhóm sử dụng:

- **k -fold cross-validation** ($k = 5$ hoặc $k = 10$): Chia data thành k phần, luân phiên train/test.
- **LOSO (Leave-One-Source-Out):** Loại bỏ một nguồn dữ liệu ra test – đánh giá khả năng tổng quát hóa sang dự án mới.

Lý do không dùng simple train/test split: Với 24 mẫu FP, split 80/20 chỉ còn ~5 mẫu test – không đại diện, kết quả không ổn định.

[C17] MMRE là gì? Nhóm dùng metric nào để so sánh các mô hình?

Trả lời: MMRE (Mean Magnitude of Relative Error):

$$MMRE = \frac{1}{n} \sum_{i=1}^n \frac{|E_{\text{actual},i} - E_{\text{predicted},i}|}{E_{\text{actual},i}}$$

MMRE < 0.25 được coi là “tốt” trong nghiên cứu ước lượng nỗ lực phần mềm (theo Conte et al.).

Bộ metric đầy đủ nhóm dùng:

- **MAE:** Sai số tuyệt đối trung bình (dễ hiểu, đơn vị = người-tháng).
- **RMSE:** Phạt nặng hơn cho sai số lớn.
- **R^2 :** Mức độ giải thích phương sai ($R^2 = 1$ là hoàn hảo).
- **MMRE:** Sai số tương đối, phổ biến trong SE estimation research.
- **PRED(25):** Tỷ lệ dự đoán có $MRE < 0.25$ – chỉ số thực tiễn.

NHÓM 5 – Kiến trúc hệ thống & AI Module

[C18] Kiến trúc tổng thể của hệ thống Insightimate là gì?

Trả lời: Hệ thống gồm ba lớp chính:

1. **Data Layer:** Thu thập, tiền xử lý, chuẩn hóa → ba file CSV (LOC/FP/UCP).
2. **AI/ML Layer:** Huấn luyện 3 mô hình (một cho mỗi schema) → lưu model file (.pkl).
3. **API Layer:** REST API nhận đầu vào từ user (LOC/FP/UCP + các tham số) → trả về Effort, Schedule, Team Size ước lượng.

Frontend (web app) tích hợp với API Layer, cho phép user chọn schema và nhập thông số dự án.

[C19] Hệ thống tích hợp role-based như thế nào? Static roles và dynamic roles khác nhau gì?

Trả lời: Static roles (4 role cố định – train trực tiếp): Được định nghĩa cứng trong hệ thống, mỗi role có hệ số nhân effort (effort multiplier) khác nhau. Ví dụ: Project Manager, Senior Developer, Junior Developer, QA Engineer.

Dynamic roles (5 người dùng – không phụ thuộc cố định): User có thể tự khai báo vai trò và kinh nghiệm; hệ thống tính toán hệ số effort dựa trên input thực tế thay vì tra bảng cố định.

Ý nghĩa: Static roles phù hợp với dự án có cơ cấu nhân sự rõ ràng; dynamic roles linh hoạt hơn cho startup và nhóm Agile nhỏ.

[C20] Hệ thống hỗ trợ Agile estimation (story points) như thế nào?

Trả lời: Đây là **hướng mở rộng** trong tương lai:

- Thu thập dữ liệu story points từ các dự án Agile (Jira export, GitHub Issues).
- Xây dựng mô hình mapping: Story Points → Effort (person-hours).
- Tích hợp vào pipeline hiện có như schema thứ 4 (SP-based).
- Kết hợp với COCOMO II calibration cho Agile: sử dụng Sprint Velocity làm hệ số điều chỉnh.

[C21] MCP (Model Context Protocol) được tích hợp như thế nào trong hệ thống?

Trả lời: MCP là giao thức kết nối AI assistant với các nguồn dữ liệu và tool bên ngoài.

Ứng dụng trong Insightimate:

- MCP tổng hợp dữ liệu thực tế từ nguồn ngoài (GitHub, Jira, Redmine) kết hợp với dữ liệu đã train.
- Cho phép AI assistant trả lời câu hỏi về dự án cụ thể dựa trên dữ liệu lịch sử.
- Hướng tích hợp: Database của tổ chức (historical project data) $\leftrightarrow$ MCP $\leftrightarrow$ ML Model $\rightarrow$ Prediction.

[C22] Metric OO (hướng đối tượng) như WMC, DIT, CBO... liên quan gì đến đề tài?

Trả lời: Các metric OO (Chidamber & Kemerer metrics) có thể được dùng như **feature bổ sung** trong schema LOC:

- **WMC** (Weighted Methods per Class): Đo độ phức tạp của lớp.
- **DIT** (Depth of Inheritance Tree): Độ phức tạp kế thừa.
- **CBO** (Coupling Between Objects): Độ kết hợp giữa các lớp.
- **LCOM** (Lack of Cohesion in Methods): Đo sự gắn kết.
- **bug**: Số lỗi thực tế – có thể dùng làm output thứ hai (bug prediction).

Các metric này giúp mô hình học được đặc điểm *chất lượng thiết kế* chứ không chỉ kích thước – là hướng mở rộng của nhóm.

NHÓM 6 – Đóng góp & Hướng tương lai

[C23] Đóng góp khoa học chính của đề tài là gì?

Trả lời:

1. **Pipeline tích hợp đa schema:** Quy trình thu thập $\rightarrow$ tiền xử lý $\rightarrow$ huấn luyện $\rightarrow$ API cho cả ba schema LOC/FP/UCP trong một hệ thống duy nhất.
2. **Chuẩn hóa dữ liệu đa nguồn:** Phương pháp luận chuẩn hóa đơn vị, xử lý outlier (IQR), log-transform có thể tái sử dụng cho nghiên cứu tương lai.
3. **So sánh thực nghiệm:** Đánh giá 3 thuật toán ML (LR, DT, RF) trên 3 schema với cùng bộ metric – cung cấp baseline cho nghiên cứu tiếp theo.
4. **Hệ thống thực tiễn:** Insightimate là web application có thể deploy thực tế, không chỉ là nghiên cứu lý thuyết.

[C24] Hạn chế của đề tài là gì? Nhóm sẽ cải thiện như thế nào?

Trả lời: Hạn chế hiện tại:

- Dataset FP (24 mẫu) và UCP còn nhỏ – ảnh hưởng độ tin cậy mô hình.
- Chưa xử lý yếu tố con người (năng suất cá nhân, kinh nghiệm team).
- Chưa tích hợp risk analysis và Monte Carlo simulation.
- Chưa hỗ trợ Agile story points.

Hướng cải thiện:

- Thu thập thêm dữ liệu FP/UCP từ ISBSG, cộng đồng học thuật.
- Tích hợp thêm LSTM cho dữ liệu time-series (effort theo sprint).
- Transfer learning: Dùng mô hình LOC (nhiều dữ liệu) để khởi tạo trọng số cho FP/UCP.
- Mở rộng schema thứ 4: Story Points (Agile).
- Tích hợp Monte Carlo simulation cho khoảng tin cậy ước lượng.

[C25] Tại sao đề tài có giá trị thực tiễn cho doanh nghiệp Việt Nam?

Trả lời:

- **Thiếu công cụ ước lượng Việt hóa:** Hầu hết công cụ COCOMO II hiện có là tiếng Anh, không phù hợp với quy trình quản lý dự án Việt Nam.
- **SME phổ biến:** 90% doanh nghiệp IT Việt Nam là SME (Small-Medium Enterprise) – không có đủ nguồn lực thuê chuyên gia ước lượng chi phí.
- **Tiết kiệm chi phí overrun:** Theo Standish Chaos Report, 52% dự án phần mềm vượt ngân sách – công cụ ước lượng tốt hơn giảm rủi ro này.
- **Ba schema đa dạng:** Hỗ trợ cả dự án truyền thống (LOC, FP) lẫn OOP/UML (UCP) – phù hợp nhiều loại dự án.

[C26] Nếu dataset FP chỉ có 24 mẫu, kết quả mô hình có đáng tin không?

Trả lời: Đây là câu hỏi quan trọng và trung thực:

Trả lời thành thật:

- Với 24 mẫu, kết quả có *giá trị tham khảo* chứ chưa đạt mức *sản xuất*.
- Nhóm sử dụng Leave-One-Out (LOO) cross-validation thay vì split thông thường để tận dụng tối đa dữ liệu.

- Confidence interval của các metric khá rộng – nhóm trình bày cả khoảng dao động, không chỉ điểm trung bình.
- Kết quả vẫn có giá trị khoa học: thiết lập **baseline** để các nghiên cứu sau cải thiện khi có thêm dữ liệu.

Lưu ý: Đây là câu hỏi mà hội đồng gần như chắc chắn sẽ hỏi. Hãy trả lời thẳng thắn và tự tin, không né tránh.

[C27] Tại sao dùng $\log_{1p}$ mà không dùng Box-Cox transform hay Yeo-Johnson?

	Transform	Ưu điểm	Nhược điểm
Trả lời:	log1p	Đơn giản, $x \geq 0$ OK, liên kết trực tiếp COCOMO	Cố định tham số
	Box-Cox	Tối ưu tham số λ	Yêu cầu $x > 0$, phức tạp
	Yeo-Johnson	Hỗ trợ cả $x < 0$	Phức tạp, khó giải thích

Lý do chọn log1p: Giá trị LOC, Effort, FP, UCP luôn ≥ 0 ; log1p an toàn với $x = 0$; kết nối trực tiếp với lý thuyết COCOMO II ($\text{Effort} = A \cdot \text{Size}^E$) – giải thích được về mặt lý thuyết.

NHÓM 7 – Câu hỏi khó / Bẫy từ hội đồng

[C28] Mô hình của nhóm có tốt hơn COCOMO II thuần không? Bằng chứng?

Trả lời: Câu trả lời có cấu trúc:

1. Mô hình COCOMO II thuần sử dụng hệ số A và E cố định từ dữ liệu lịch sử toàn cầu – không hiệu chỉnh theo tổ chức/domain cụ thể.
2. ML model của nhóm **học** từ dữ liệu thực tế – tự hiệu chỉnh tham số.
3. Theo nghiên cứu (Jørgensen & Shepperd, 2007): ML-based models thường giảm MMRE 15–30% so với COCOMO II thuần trên cùng dataset.
4. Nhóm so sánh trực tiếp: chạy COCOMO II với tham số mặc định và RF model trên cùng test set – RF đạt MMRE thấp hơn.

[C29] Nếu dữ liệu train và test đến từ cùng một nguồn, kết quả có bị overfitting không?

Trả lời: **Rủi ro có thật.** Nhóm giảm thiểu bằng:

- Thêm cột **source** và thực hiện **LOSO cross-validation** (Leave-One-Source-Out): model train trên tất cả nguồn trừ một, test trên nguồn còn lại.
- LOSO kiểm tra khả năng tổng quát hóa sang dữ liệu từ tổ chức hoàn toàn mới.
- Kết quả LOSO so sánh với k -fold để phát hiện nếu có data leakage.

[C30] Scatter plot cho thấy mối quan hệ phi tuyến – tại sao Linear Regression vẫn được giữ?

Trả lời: Linear Regression được giữ làm **baseline** (điểm chuẩn):

- Cung cấp kết quả so sánh “tối thiểu” – nếu RF không tốt hơn LR, có vấn đề trong quy trình.
- Trong không gian log-log, quan hệ Effort–Size trở thành tuyến tính $\rightarrow$ LR trên log-transformed data là xấp xỉ COCOMO tuyến tính.
- LR interpretable hoàn toàn: hệ số hồi quy tương ứng trực tiếp với tham số COCOMO A và E .

[C31] Làm sao đảm bảo hệ thống không bị “garbage in, garbage out” khi user nhập sai?

Trả lời: Nhóm xây dựng **validation layer** ở API:

- Kiểm tra kiểu dữ liệu và range hợp lệ ($LOC > 0$, $Effort > 0$).
- Cảnh báo nếu giá trị đầu vào nằm ngoài phạm vi training data (extrapolation warning).
- Yêu cầu chọn schema rõ ràng trước khi dự đoán.
- Log đầu vào để audit và cải thiện mô hình theo thời gian.

TÓM TẮT CÁC ĐIỂM CẦN NHỚ KHI BẢO VỆ

- ✓ COCOMO II: $\text{Effort} = A \times \text{Size}^E \times \prod EM_i$
- ✓ Ba schema: LOC (947 mẫu), FP (24 mẫu), UCP
- ✓ IQR: $\text{clip}(Q1 - 1.5 \times \text{IQR}, Q3 + 1.5 \times \text{IQR})$ – giữ số lượng mẫu
- ✓ log1p vì: an toàn với 0, liên kết COCOMO lý thuyết
- ✓ Random Forest > Deep Learning: dataset nhỏ, cần interpretability
- ✓ LOSO CV để kiểm tra tổng quát hóa qua nguồn dữ liệu
- ✓ FP ít mẫu $\rightarrow$ kết quả tham khảo, trình bày confidence interval
- ✓ Hướng tương lai: Story Points, LSTM, Transfer Learning, MCP